Conformance Checking for Process Improvement

JM0211 Process Mining

Group 8: Rick de Rijk, Thom Verzantvoort, Stefan Vonk, Tycho van Rooij

Table of contents

1 Motivation and Scientific Background **2**

2 Conceptual Approach **2**
- 2.1 Overview . . . 2
- 2.2 Step 1: KPI definition and enrichment . . . 3
- 2.3 Step 2: Alignment and encoding . . . 3
- 2.4 Step 3: Classification . . . 4
- 2.5 Step 4: Rules and trace subsets . . . 4

3 Implementation **4**

4 Evaluation Design and Results **5**
- 4.1 Data and evaluation design . . . 5
- 4.2 Gold standards for alignments and rules . . . 5
- 4.3 Results on the real log . . . 5

5 Limitations and Future Work **7**

6 References **7**

Appendix **8**
- Lifecycle merge experiment . . . 8

List of Figures

1 The artefact turns a log and a model into rules in four steps. . . . 3
2 F1 of the slow class for each cutoff, compared with the baseline that labels every trace as slow. Error bars show one standard deviation over four seeds. . . . 6

1 Motivation and Scientific Background

Conformance checking compares behaviour recorded in an event log with behaviour accepted by a process model in order to identify commonalities and deviations (Aalst 2016). Alignment-based conformance checking sees deviations as *log moves*, representing observed behaviour that cannot be reproduced by the model, and *model moves*, representing modelled behaviour that was not observed. The course topics on resource and temporal conformance also check conformance, but against constraints from natural-language descriptions, whereas we compare control flow with a Petri net.

Deviations from a normative process model are not directly linked with poor process performance. Dees et al. (Dees, Leoni, and Mannhardt 2017) propose a methodology in which deviations identified through conformance checking are related to a case-level KPI using classification or regression trees, and deviation patterns associated with good KPI values can help to enhance the model. The method complements decision mining: where decision mining uses trees and rules to explain the routing at decision points, we explain a case-level KPI by the deviations from the model. Predictive process monitoring also links process features to outcomes, but it predicts them for ongoing cases, whereas our rules are interpretable by design and aim to improve the process.

This assignment focuses on the deviation analysis component of this method. For each case, optimal alignments against a Petri net are used to derive deviation features, which are related to case throughput time. Throughput time is defined as the elapsed time between the first and last event of a trace, following the KPI definition used in the UWV case study of Dees et al. (Dees, Leoni, and Mannhardt 2017).

As a concrete use case, the loan application process of a Dutch financial institution from the BPI Challenge 2017 (Dongen 2017) is taken, restricted to the denied applications. These applications take between less than a day and about 260 days from the first to the last event. A process owner may ask which deviations from the process go together with slow denials, and whether the process should allow or forbid them. This is the question that the deviation analysis answers. The events contain lifecycle information, which the alpha miner topic also handles and which helps to shape our features (Section 5).

Related work covers parts of this setting. Recent work lifts event-level deviations to process-level patterns, such as activities that are inserted, skipped, repeated, replaced or swapped (Grohs, Aa, and Rehse 2024). The BPIC studies that we reviewed do not use the link between deviations and a KPI. Carvallo et al. (Carvallo et al. 2017) analyse throughput times descriptively, reporting a mean of 18.1 days for approved applications. They leave denied applications out of their deeper analysis. Ryu et al. (Ryu et al. 2017) apply decision trees at the decision points of the process, but obtain usable results at only one of three. Our modelling choices follow common practice in tree-based process mining: Park et al. (Park et al. 2023) select decision-tree parameters by grid search, compare models with five-fold cross-validation and the F1 score, and prune trees to keep them interpretable. To our knowledge, none of these studies relates alignment-based deviation features to a throughput KPI for the Denied subset, which is the gap that this work addresses.

2 Conceptual Approach

2.1 Overview

The artefact implements the deviation analysis step of Dees et al. (Dees, Leoni, and Mannhardt 2017). It takes an event log in XES format and a Petri net in PNML format as input. The log must provide a case notion, an activity label for every event and a reliable timestamp for every event, because both the throughput time and the alignments depend on them. Resources and further attributes are not used. The Petri net is the normative model against which the log is compared. The artefact returns a set of classification rules that relate deviations to throughput time, together with the traces of the log that satisfy each rule.

Figure 1 shows the four steps, which the following subsections describe in order: the KPI, the alignment and encoding, the classification tree and the rule extraction. The parameters are the throughput cutoff (required), the hold-out fraction, the random seed and an optional sample size for development. The correctness of the implementation is tested against hand-built gold standards, which Section 4 describes.

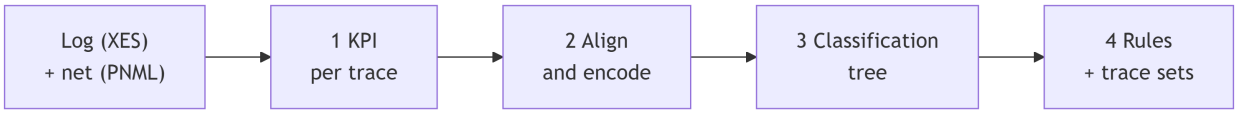


Figure 1: The artefact turns a log and a model into rules in four steps.

2.2 Step 1: KPI definition and enrichment

The first step attaches a key performance indicator (KPI) to every trace, so that deviations can later be related to how long a case takes. The brief prescribes throughput time. For a trace σ, it is the elapsed time between the first and the last event (Dees, Leoni, and Mannhardt 2017):

$$\kappa(\sigma) = t(\text{last event of } \sigma) - t(\text{first event of } \sigma).$$

The artefact stores $\kappa(\sigma)$ in days as the case attribute `throughput_time`, and lower values are better. The implementation takes the earliest and the latest timestamp of the trace, which equals the definition for chronologically ordered traces. An empty trace or an event without timestamp raises an error, and a trace with a single event receives a throughput time of zero.

2.3 Step 2: Alignment and encoding

The second step measures how each trace deviates from the Petri net. An alignment maps a trace onto a run of the net, and every position of the alignment is a synchronous move, a log move or a model move. A synchronous move is an activity that the trace and the model perform together. A log move is an activity that the trace contains and the model does not allow at that point. A model move is an activity that the model requires and the trace lacks (Aalst 2016). Table 1 shows the three move types for a short trace.

Table 1: Log moves and model moves arise where a trace and the model disagree. The table shows one alignment of the trace A, A, C against the model A, B, C, in which the symbol ≫ marks a skipped position.

Position	1	2	3	4
Trace	A	A	≫	C
Model	A	≫	B	C
Move	synchronous	log move A	model move B	synchronous

An alignment is optimal when it has the minimal number of deviations on visible transitions, and moves on silent transitions do not count (Dees, Leoni, and Mannhardt 2017). The alignments are computed with PM4Py (Berti, Zelst, and Aalst 2019). In the version used (2.7.23.8), the default cost function and the state-equation A* solver realise this criterion: synchronous moves cost 0, log moves and model moves on visible activities cost 10000 each, and moves on silent transitions cost 1, which serves only as a tie-breaker. A trace can have several optimal alignments, and the solver returns one of them. Dees et al. argue that this choice does not affect the applicability of the methodology when the log contains many traces (Dees, Leoni, and Mannhardt 2017). Section 5 returns to this assumption.

Each alignment is then encoded as a vector of counts. For each of the 54 visible activities of the net, one feature records the number of model moves and one the number of log moves of that activity, as the brief specifies. This gives 108 features per trace. Dees et al. exclude the activities that analysts declare disallowed or mandatory from these features (Dees, Leoni, and Mannhardt 2017), but the brief provides no such constraints, so all activities are used. An activity is an event name including its lifecycle suffix, for example `W-Call-after-offers-suspend`. Synchronous moves are not features, because they carry no deviation. Silent transitions are not features either, because they never appear in the log. The counts

replace the activity sequence as representation, since the 3,093 traces contain 2,038 distinct sequences and the sequence itself is therefore an unsuitable feature.

2.4 Step 3: Classification

The third step relates the encoded deviations to the KPI with a classification tree. Dees et al. use classification trees for discrete and regression trees for numeric KPI values (Dees, Leoni, and Mannhardt 2017). The brief asks for a classification tree, so a trace is slow if its throughput time exceeds a cutoff of 14 days, and fast otherwise. The cutoff plays the role of the set of satisfactory KPI values in Dees et al., who accept every duration below 8 days in their UWV case study (Dees, Leoni, and Mannhardt 2017). It lies close to the median of 14.43 days, yields 1,501 fast and 1,592 slow traces, and was fixed before any tree was fitted. A rule therefore describes deviations that go with a throughput time above or below this cutoff. It does not show that they cause the delay or how large the delay is, and a different cutoff can give different rules, so Section 4 compares three cutoffs. A regression tree would retain the magnitude that the discretisation discards.

The tree is fitted on a stratified random sample of 75 percent of the traces, and the remaining 25 percent is held out for evaluation. A grid search with five-fold cross-validation on the training part selects the maximum depth (2 to 5), the minimum leaf size (5, 10 or 20) and the cost-complexity pruning parameter (Breiman et al. 1984) (0, 0.001, 0.01 or 0.1). This gives 48 combinations, and the grid was fixed before any scores were inspected. The selection criterion is the F1 score of the slow class. Pruning limits the number of rules. Unpruned trees produced at least twenty rules, which is too many to interpret.

2.5 Step 4: Rules and trace subsets

The fourth step reads the fitted tree as rules. Every path from the root to a leaf is a conjunction of threshold conditions on move counts, and each leaf is one rule. The counts are integers, so a condition such as `log_move_X <= 0.5` means that activity X never occurs as a log move, and `log_move_X > 0.5` means that it occurs at least once. A rule predicts the class that the tree assigns to its leaf. For each rule, the artefact lists the identifiers of all traces that satisfy its conditions, including those whose actual class differs from the predicted one. Dees et al. keep only correctly classified traces in a cluster (Dees, Leoni, and Mannhardt 2017). The rules and trace lists cover all 3,093 traces, whereas precision and recall are computed on the hold-out set only.

3 Implementation

The artefact is implemented in Python 3.12 and organised in three packages that mirror the pipeline: data loading with KPI enrichment, alignment with encoding, and classification with rule extraction. PM4Py (Berti, Zelst, and Aalst 2019) (version 2.7.23.8) loads the XES log and the PNML net and computes the alignments. The package pandas holds the encoding table, and scikit-learn (Pedregosa et al. 2011) fits the tree and performs the grid search. A single entry point, `src/main.py`, runs all four steps. It requires the cutoff in days and accepts the hold-out fraction, the random seed, an optional sample size and the output paths as optional parameters. The random seed defaults to 0, so that a run is reproducible. The program writes the encoding table, the rule table and the rule-to-trace assignments as CSV files, and it can export the log enriched with throughput time as XES. All paths are relative to the project root, and `requirements.txt` pins the dependencies. Scripts in the `scripts` directory generate the two toy logs and nets, compare their alignments and rules with the hand-written gold standards, and compare the cutoffs and seeds on the real log (`compare_cutoffs.py`). With `--merge`, that script sums the suspend and resume counts of each task into a pause feature and the schedule, start and complete counts into an execution feature (108 to 74 features), without recomputing the alignments. A second script, `compare_cutoffs_merge_all.py`, sums all lifecycle variants of a task into one feature (46 features).

4 Evaluation Design and Results

4.1 Data and evaluation design

The evaluation uses the Denied subset of the BPIC 2017 log (Dongen 2017) and the Petri net provided with the assignment. The subset has 3,093 traces over 54 visible activities, and throughput times between 0.006 and 259.9 days, with a median of 14.43 and a mean of 17.0 days. A sample of the log served only during development, and all reported numbers use the full log.

The brief asks how well the approach generates the optimal alignments and how well it finds the right classification rules. We therefore evaluate in two ways. The correctness of the alignments and of the rules is tested on two small artificial logs against hand-built gold standards (Section 4.2). On the real log no gold standard for the rules exists, so we measure how well the tree separates slow from fast traces on a stratified hold-out of 25 percent of the traces, with the precision, recall and F1 of the slow class. Each result is compared with the baseline that labels every trace as slow, which has a precision equal to the share of slow traces, a recall of 1 and an F1 of $2p/(1+p)$ for slow share p. The main setting is the cutoff of 14 days. Cutoffs of 18 and 21 days, with slow shares of 33.4 and 24.8 percent, test how sensitive the results are to this choice. At these cutoffs the slow class is a minority, so we also compare uniform class weights with balanced weights. These two variants are a sensitivity analysis and not part of the grid that was fixed in advance. A single random split can mislead, so every setting is run with the seeds 0 to 3, and we report the mean and the standard deviation. The seeds reuse the same 3,093 traces, so the standard deviation measures the sensitivity to the split and not independent replication.

4.2 Gold standards for alignments and rules

The correctness of the alignments and the rules is tested against gold standards built by hand, because the traces of the real log are too long to align manually. Two small logs with their nets serve this purpose: a linear net with five traces, and a net with six traces that adds a parallel block and an exclusive choice. The correct move counts of every trace were derived by hand before the artefact was run. For the rules, the throughput times of selected traces were planted so that one deviation separates slow from fast traces, and the recovered trace sets are compared with the planted ones. Both toys reproduce the hand-derived move counts exactly (5 of 5 and 6 of 6 traces). Counting every move of the gold standard as one item, the aligner finds all gold moves and adds none, so the precision and the recall of the alignments are 1.0 on both toys. Both planted rules are recovered with precision and recall of 1.0.

4.3 Results on the real log

At the cutoff of 14 days, the tree with seed 0 has two rules, a precision of 0.563, a recall of 0.970 and an F1 of 0.712 (Table 2). Averaged over four seeds, the F1 of 0.688 is only slightly above the 0.680 of the baseline, and the selected tree has between 2 and 14 rules. Seeds 0 and 3 give the two-rule tree, whereas seeds 1 and 2 give trees with 11 and 14 rules and a recall of about 0.74. The only split of the two-rule tree is the log move of `W-Call-after-offers-suspend`. The net allows one suspension of this task, so a log move means an additional suspension. The 389 traces without it are slow in 16 percent of the cases, and the 2,704 traces with it in 57 percent. The larger leaf holds 87 percent of the traces, and its slow share is only 5 points above the base rate of 51.5 percent, so the association is modest.

At the higher cutoffs the slow class is a minority, and the two class weights show a trade-off (Table 3, Figure 2). Uniform weights give a precision of 0.48 and 0.61, clearly above the base rates of 0.33 and 0.25, but a recall of only 0.17 and 0.13. Balanced weights reach a recall of about 0.8 and an F1 above the baseline (0.540 against 0.501, and 0.460 against 0.397), but their precision is only about 7 points above the base rate. When the grid search was allowed to choose the class weight, it selected the balanced weight in all four seeds at both cutoffs. In the trees with seed 0, the clearest slow rules at 18 and 21 days involve repeated log moves of `O-Refused`: three or more at 18 days, in leaves of 29 to 55 traces with 59 to 71 percent slow traces, and at 21 days in combination with `O-Created`, in a leaf of 113 traces with 60 percent slow traces.

We repeated the comparison with two merged encodings (Section 3, Table 4 in the appendix). No F1 differs from its unmerged value by more than one standard deviation, for example 0.692 against 0.688 at 14 days with uniform weights and 0.540 against 0.540 at 18 days with balanced weights. The root split is the same, since the pause feature of `W-Call-after-offers` selects the same 389 and 2,704 traces. The rule counts do not become more stable: at 14 days the merged tree for seed 1 shrinks to a single rule that labels every trace as slow. Merging all lifecycle variants of a task gives the same two-rule tree in every seed at 14 days with uniform weights (F1 0.713 ± 0.011), but not at the other cutoffs.

Table 2: Cutoff of 14 days. Hold-out scores of the slow class.

Model	Precision	Recall	F1	Rules
All slow (baseline)	0.515	1.000	0.680	1
Tree, seed 0	0.563	0.970	0.712	2
Tree, seeds 0 to 3	0.582 ± 0.019	0.855 ± 0.133	0.688 ± 0.030	7.3 (2 to 14)

Table 3: Cutoffs of 18 and 21 days. Hold-out scores of the slow class, mean ± standard deviation over four seeds.

Cutoff (slow share)	Weights	Precision	Recall	F1	Baseline F1	Rules
18 days (33.4%)	uniform	0.483 ± 0.104	0.171 ± 0.046	0.247 ± 0.042	0.501	12.3 (4–22)
18 days (33.4%)	balanced	0.403 ± 0.016	0.826 ± 0.106	0.540 ± 0.009	0.501	8.0 (2–12)
21 days (24.8%)	uniform	0.608 ± 0.021	0.133 ± 0.020	0.217 ± 0.027	0.397	11.0 (8–18)
21 days (24.8%)	balanced	0.320 ± 0.014	0.814 ± 0.030	0.460 ± 0.018	0.397	12.3 (11–14)



Figure 2: F1 of the slow class for each cutoff, compared with the baseline that labels every trace as slow. Error bars show one standard deviation over four seeds.

Two settings fail. With uniform weights at 18 and 21 days, the F1 falls far below the baseline, because the largest leaves are mostly fast and the tree predicts slow only in small leaves of 24 to 113 traces. At 14 days, the same grid selects two rules for two seeds and 11 to 14 rules for the other two, so the two-rule tree is not a stable explanation. The experiments support that the tree beats the all-slow baseline only slightly at 14 days (0.688 against 0.680), and by 0.04 to 0.06 with balanced weights at 18 and 21 days. They do not

show why: we did not test whether move counts carry little information about throughput time or whether the tree and the grid are too restrictive. Class weights change the F1 far more than merging the lifecycle variants does, so the model settings matter more than the feature encoding.

5 Limitations and Future Work

The approach has several design limitations. Discretising throughput time at one cutoff discards the magnitude of the KPI, whereas Dees et al. use a regression tree for the numeric KPI of their UWV case (Dees, Leoni, and Mannhardt 2017). Throughput time is elapsed wall-clock time, so it includes non-working periods and does not separate waiting from processing. An optimal alignment is also not unique, and the features depend on the one that the solver returns. Dees et al. argue that this choice does not matter when the log contains many traces (Dees, Leoni, and Mannhardt 2017), but this was not tested here.

Two further limitations concern the encoding and the rule output. Activities are lifecycle events, such as the suspension or the abortion of a work item, so many features describe interruptions and are hard to read as process deviations. Merging the lifecycle variants did not change the scores, apart from stabilising the 14-day tree when all variants are merged (Section 4.3), and it did not make the rules easier to interpret. The trace subset of a rule contains every trace that satisfies its conditions, including misclassified ones, whereas Dees et al. keep only correctly classified traces (Dees, Leoni, and Mannhardt 2017).

The results add limitations, of which three weigh most for the reliability of the findings. First, at the cutoff of 14 days the tree improves the F1 of the all-slow baseline by only 0.008 on average over four seeds, and the selected tree has between 2 and 14 rules, so the two-rule tree is not a stable explanation. Second, the evaluation covers a single dataset, with four splits of the same traces, and no published BPIC 2017 study with the same setup exists for a comparison. Third, nearly every trace deviates (median 8 deviations; log moves of `A-Denied`, `O-Refused` and `O-Returned` in over 90 percent of the traces), so deviations are the norm and not exceptions. The root split, a log move on `W-Call-after-offers-suspend`, occurs in 87 percent of the traces and its slow share is only 5 points above the base rate, so it may reflect a net that is too strict about suspensions. Less critical are the trade-off between precise rules with low recall and high recall with weak precision at higher cutoffs, and the use of control flow only, since the topic description also names violations of data guards, such as the loan amount.

Future work follows from these limitations. A net that allows repeated suspensions, is rediscovered from the log, or is built from the fastest training traces would make the second suspension normal, if the given net may be changed. Time-aware features, such as the days waited after a suspension, would capture the waiting that alignments ignore. Comparing only the fastest and slowest quarter, or a regression tree on the continuous KPI, would avoid the arbitrary cutoff. Sampling several optimal alignments would test the solver's influence.

6 References

Appendix

Lifecycle merge experiment

We tested whether the encoding of lifecycle events affects the results. The unmerged encoding has one feature per lifecycle variant (108 features). The pause/execution encoding sums the suspend and resume counts of each work task into one pause feature and the schedule, start and complete counts into one execution feature (74 features). The all merged encoding sums all lifecycle variants of a task, including `-ate-abort` and `-withdraw`, into one feature per move kind (46 features). The A- and O- activities are unchanged and the alignments are not recomputed. All encodings use the same cutoffs, seeds 0 to 3, grid and class weights. Table 4 shows the hold-out scores as mean ± standard deviation over four seeds, and the number of rules as mean (minimum to maximum). Pause/execution changes no F1 by more than one standard deviation; its 14-day uniform tree with seed 1 is a single rule that labels every trace as slow. All merged gives the same two-rule tree in every seed at 14 days with uniform weights (F1 0.713 ± 0.011), but it does not improve the other settings, and balanced weights at 21 days are slightly lower (0.444 against 0.460). We therefore do not consider any encoding better.

Table 4: Three lifecycle encodings. Hold-out scores of the slow class, mean ± standard deviation over four seeds.

Cutoff	Weights	Encoding	Precision	Recall	F1	Rules
14 days	uniform	unmerged	0.582 ± 0.019	0.855 ± 0.133	0.688 ± 0.030	7.3 (2–14)
14 days	uniform	pause/execution	0.560 ± 0.034	0.921 ± 0.119	0.692 ± 0.026	4.8 (1–14)
14 days	uniform	all merged	0.571 ± 0.009	0.948 ± 0.017	0.713 ± 0.011	2.0 (2–2)
14 days	balanced	unmerged	0.584 ± 0.023	0.848 ± 0.141	0.687 ± 0.032	7.0 (2–13)
14 days	balanced	pause/execution	0.583 ± 0.021	0.854 ± 0.134	0.689 ± 0.030	5.3 (2–10)
14 days	balanced	all merged	0.579 ± 0.022	0.890 ± 0.114	0.698 ± 0.026	4.3 (2–11)
18 days	uniform	unmerged	0.483 ± 0.104	0.171 ± 0.046	0.247 ± 0.042	12.3 (4–22)
18 days	uniform	pause/execution	0.500 ± 0.074	0.184 ± 0.042	0.264 ± 0.034	16.8 (4–21)
18 days	uniform	all merged	0.500 ± 0.053	0.169 ± 0.014	0.252 ± 0.020	13.5 (9–18)
18 days	balanced	unmerged	0.403 ± 0.016	0.826 ± 0.106	0.540 ± 0.009	8.0 (2–12)
18 days	balanced	pause/execution	0.403 ± 0.016	0.826 ± 0.106	0.540 ± 0.009	8.8 (2–12)
18 days	balanced	all merged	0.404 ± 0.015	0.808 ± 0.113	0.536 ± 0.009	8.0 (2–12)
21 days	uniform	unmerged	0.608 ± 0.021	0.133 ± 0.020	0.217 ± 0.027	11.0 (8–18)
21 days	uniform	pause/execution	0.592 ± 0.045	0.124 ± 0.017	0.204 ± 0.022	11.8 (8–22)
21 days	uniform	all merged	0.577 ± 0.039	0.130 ± 0.027	0.211 ± 0.033	11.5 (7–20)

Cutoff	Weights	Encoding	Precision	Recall	F1	Rules
21 days	balanced	unmerged	0.320 ± 0.014	0.814 ± 0.030	0.460 ± 0.018	12.3 (11–14)
21 days	balanced	pause/execution	0.320 ± 0.014	0.814 ± 0.030	0.460 ± 0.018	11.5 (10–14)
21 days	balanced	all merged	0.332 ± 0.034	0.712 ± 0.163	0.444 ± 0.023	13.3 (8–19)

Aalst, Wil M. P. van der. 2016. *Process Mining: Data Science in Action*. 2nd ed. Springer. https://doi.org/10.1007/978-3-662-49851-4.

Berti, Alessandro, Sebastiaan J. van Zelst, and Wil M. P. van der Aalst. 2019. "Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science." In *Proceedings of the ICPM Demo Track 2019*, 2374:13–16. CEUR Workshop Proceedings. https://arxiv.org/abs/1905.06169.

Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. 1984. *Classification and Regression Trees*. Belmont, CA: Wadsworth.

Carvallo, Andrés, Cristóbal Henning, Dasen Razmilic, Regina Reyes López, Jonathan Lee, Juan P. Salazar Fernández, and Michael Arias. 2017. "Applying Process Mining for Loan Approvals in a Banking Institution." BPI Challenge 2017 report.

Dees, Marcus, Massimiliano de Leoni, and Felix Mannhardt. 2017. "Enhancing Process Models to Improve Business Performance: A Methodology and Case Studies." In *On the Move to Meaningful Internet Systems. OTM 2017 Conferences*, 10573:232–51. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-319-69462-7_15.

Dongen, B. F. van. 2017. "BPI Challenge 2017." 4TU.ResearchData. https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b.

Grohs, Michael, Han van der Aa, and Jana-Rebecca Rehse. 2024. "Beyond Log and Model Moves in Conformance Checking: Discovering Process-Level Deviation Patterns." In *International Conference on Business Process Management*, 381–99. Springer. https://doi.org/10.1007/978-3-031-70396-6_22.

Park, Gyunam, Aaron Küsters, Mara Tews, Cameron Pitsch, Jonathan Schneider, and Wil M. P. van der Aalst. 2023. "Explainable Predictive Decision Mining for Operational Support." In *Service-Oriented Computing – ICSOC 2022 Workshops*, 13821:66–79. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-031-26507-5_6.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. "Scikit-Learn: Machine Learning in Python." *Journal of Machine Learning Research* 12: 2825–30.

Ryu, Dohyeon, Jitaek Lim, Dawoon Jeong, and Minseok Song. 2017. "Analysis of Loan Process Using Various Process Mining Techniques: The BPIC 2017." BPI Challenge 2017 report.